

4주차

딥러닝 파이토치 5장

5.3 전이학습

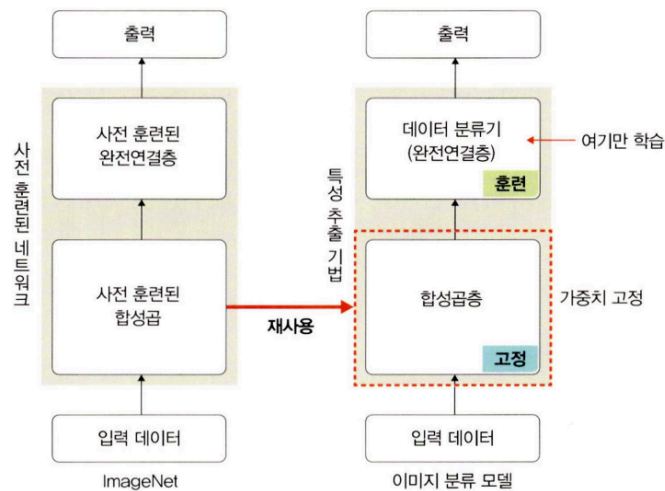
전이학습

- 아주 큰 데이터셋 사용 → 훈련된 모델의 가중치 가져옴 → 해결하려는 과제에 맞게 조정해서 사용
- 사전 훈련된 모델(네트워크): 아주 큰 데이터셋을 사용해 훈련된 모델
- 비교적 적은 수의 데이터로도 해결 가능

5.3.1 특성 추출 기법

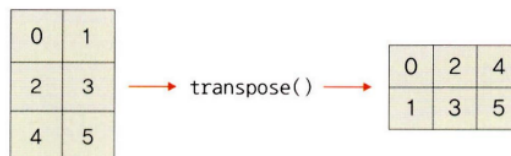
- 과정
 1. ImageNet 데이터셋으로 사전 훈련된 모델 가져옴
 2. 마지막에 완전연결층 부분만 새로 만들

⇒ 학습할 때는 마지막 완전연결층만 학습하고 나머지 계층은 안함
- 구성
 - 합성곱층: 합성곱층 + 풀링층
 - 데이터 분류기(완전연결층): 추출된 특성 입력 받음 → 최종적으로 이미지에 대한 클래스 분리
- 이미지 분류 모델
 - Xception
 - Inception V3
 - ResNet50
 - VGG16
 - VGG19
 - MobileNet



RandomResizedCrop

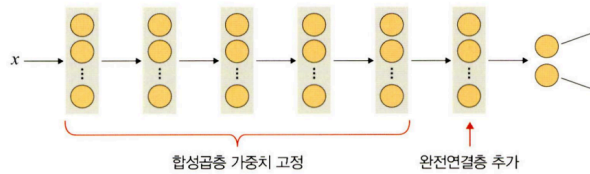
- iterator
 - `iter()` : 전달된 데이터의 반복자를 꺼내 반복
 - `next()` : 그 반복자가 다음에 출력해야 할 요소 반환
- `transpose()` : 행렬 차원 변경 (by 내적)



만약 두 행렬의 열의 수가 다르면, `np.transpose()` 나 `np.reshape()` 로 차원 조정 필요

ResNet18

- 50개층으로 구성된 합성곱 신경망
- 전이 학습에 사용되도록 사전 훈련된 모델 제공함
- 단점
 - 입력 제약 큼
 - 충분한 크기의 RAM 없으면 학습속도 느림
- 마지막 부분에 완전 연결층 추가



사전 훈련된 모델

- 방법 1: 무작위 가중치로 모델 구성

```
import torchvision.models as models
resnet18 = models.resnet18()
alexnet = models.alexnet()
vgg16 = models.vgg16()
squeezenet = models.squeezenet1_0()
densenet = models.densenet161()
inception = models.inception_v3()
googlenet = models.googlenet()
shufflenet = models.shufflenet_v2_x1_0()
mobilenet_v2 = models.mobilenet_v2()
mobilenet_v3_large = models.mobilenet_v3_large()
mobilenet_v3_small = models.mobilenet_v3_small()
resnext50_32x4d = models.resnext50_32x4d()
wide_resnet50_2 = models.wide_resnet50_2()
mnasnet = models.mnasnet1_0()
```

- 방법 2: 사전 학습된 모델의 가중치 값 사용

`pretrained=True` 적

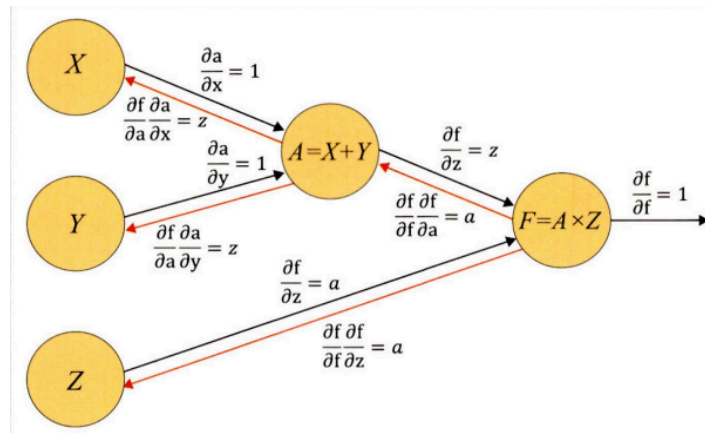
```
import torchvision.models as models
resnet18 = models.resnet18(pretrained=True)
alexnet = models.alexnet(pretrained=True)
squeezenet = models.squeezenet1_0(pretrained=True)
vgg16 = models.vgg16(pretrained=True)
densenet = models.densenet161(pretrained=True)
```

```
inception = models.inception_v3(pretrained=True)
googlenet = models.googlenet(pretrained=True)
shufflenet = models.shufflenet_v2_x1_0(pretrained=True)
mobilenet_v2 = models.mobilenet_v2(pretrained=True)
mobilenet_v3_large = models.mobilenet_v3_large(pretrained=True)
mobilenet_v3_small = models.mobilenet_v3_small(pretrained=True)
resnext50_32x4d = models.resnext50_32x4d(pretrained=True)
wide_resnet50_2 = models.wide_resnet50_2(pretrained=True)
mnasnet = models.mnasnet1_0(pretrained=True)
```

tensor 함수 비교

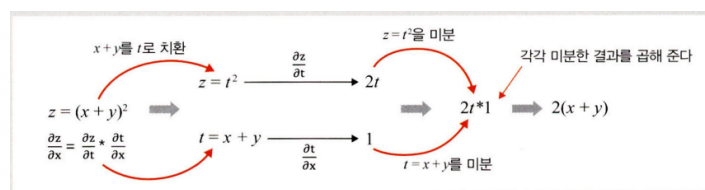
구분	메모리	계산 그래프 상주 유무
tensor.clone()	새롭게 할당	계산 그래프에 계속 상주
tensor.detach()	공유해서 사용	계산 그래프에 상주X
tensor.clone().detach()	새롭게 할당	계산 그래프에 상주X

계산 그래프



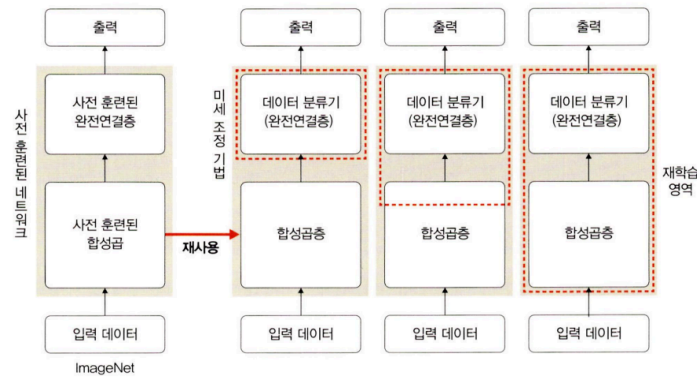
- 국소적계산 가능
- 역전파를 통한 미분 계산 편리

연쇄 법칙



- 두개 이상의 함수가 결합된 함수
- 합성 함수의 미분법: 연쇄 법칙, chain rule

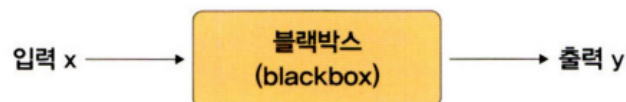
5.3.2 미세 조정 기법



- 특성 추출 기법에서 더 나아가 사전 훈련된 모델과 합성곱층, 데이터 분류기의 가중치를 업데이트해서 훈련시키는 방식
- 특성 추출: 목표 특성을 잘 추출해야 좋은 성능을 낼 수 있음
- 주의할점: 파라미터에 큰 변화를 주게되면 overfitting 문제가 발생할 수 있
- 전략
 - 데이터셋이 크고 사전 훈련된 모델과 유사성이 작을 경우: 모델 전체 재학습시킴
 - 데이터셋이 크고 사전 훈련된 모델과 유사성이 클 경우: 합성곱층의 뒷부분과 데이터 분류기를 학습시킴
 - 데이터셋이 작고 사전 훈련된 모델과 유사성이 작을 경우: 합성곱층의 일부분과 데이터 분류기를 학습시킴
 - 데이터셋이 작고 사전 훈련된 모델과 유사성이 클 경우: 데이터 분류기만 학습시킴

5.4 설명 가능한 CNN

- 딥러닝 처리 결과를 사람이 이해할 수 있는 방식으로 제시하는 기술
- CNN은 블랙박스처럼 내부에서 어떤 동작이 일어나는지 설명하기 어려움 ⇒ CNN 처리 과정 시각화 필요



5.4.1 특성 맵 시각화

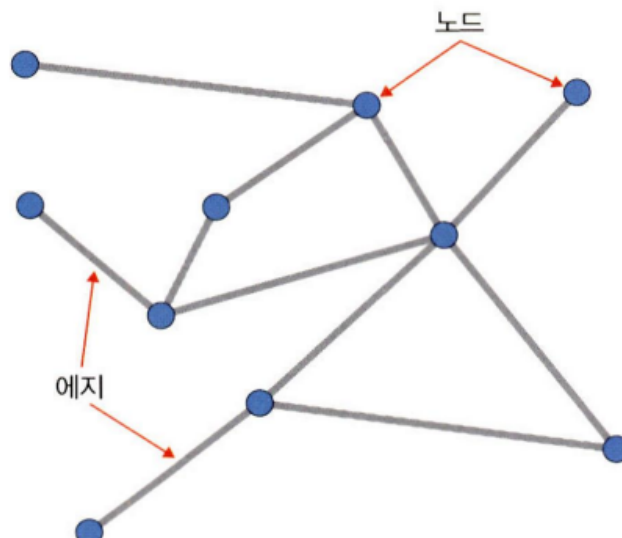
- feature map: 입력 이미지/다른 특성 맵처럼 필터를 입력에 적용한 결과

- 특성 맵에서 입력 특성을 감지하는 방법을 이해할 수 있도록 도움
- 출력층에 가까울수록 이미지 특징들만 전달됨

5.5 그래프 합성곱 네트워크

그래프 데이터를 위한 신경망

5.5.1 그래프란



- 노드(node, vertex): 그림에서 파란색 원
- 엣지(edge): 두 노드를 연결한 선. 결합 방법(single, double, triple, aromatic 등)을 의미

5.5.2 그래프 신경망

표현 단계



1. 인접 행렬 (adjacency matrix)

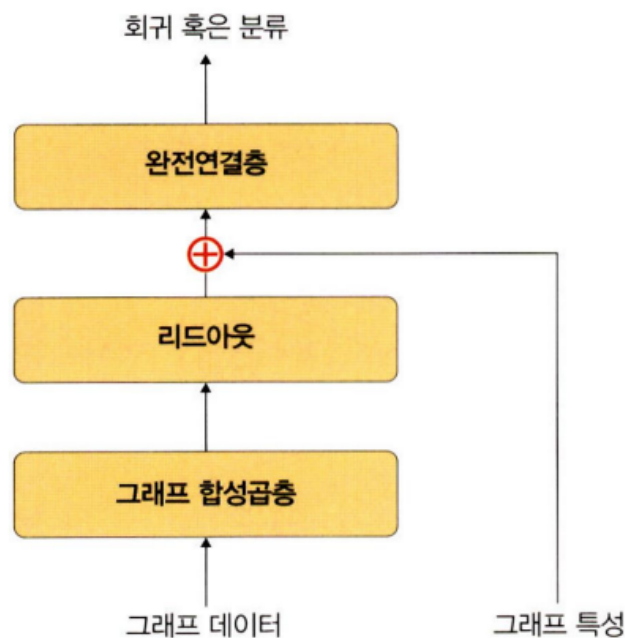
- 노드 n 개를 $n \times n$ 행렬로 표현
- A_{ij} 는 i 와 j 의 관련성 여부를 만족하는 값으로 채움

2. 특성 행렬(feature matrix)

- 인접 행렬만으로는 특성 파악 어려움 $\Rightarrow$ 단위 행렬 적용
- 각 입력 데이터에서 이용할 특성 선택
- 행: 선택된 특성에 대해 각 노드가 갖는 값
- 특성 행렬을 거쳐 그래프 특성이 추출됨

5.5.3 그래프 합성곱 네트워크 (GCN)

- 이미지에 대한 합성곱을 그래프 데이터로 확장한 알고리즘
- 구조



- 리드아웃: 특성 행렬을 하나의 벡터로 변환하는 함수
 1. 전체 노드의 특성 벡터에 대해 평균 구함
 2. 그래프 전체를 표현하는 하나의 벡터 생성
- 합성곱층
 - 행렬 형태 $\Rightarrow$ 딥러닝 알고리즘 적용 가능
 - 활용 분야
 - SNS: 관계 네트워크
 - 학술 연구: 인용 네트워크

- 3D MESH